AIRPORTS AUTHORITY OF INDIA

Search and Rescue Decision Support System

(SAR-DSS)

A GIS-based software tool that aids Search and Rescue operations by calculating and plotting the probable search area of a missing aircraft from its Last Known Position (LKP).

Deployment	URL
Frontend (Vercel)	https://client-sooty-tau-63.vercel.app
Backend API (Railway)	https://api-production-1fa39.up.railway.app/api/v1
API Documentation	https://api-production-1fa39.up.railway.app/docs
Source Code (GitHub)	https://github.com/vijinw340-bit/sar-dss

Date: 08 October 2026 | Version 1.0 | Status: Live in production

1. Project Objective

Develop a modern GIS-based web application that assists Search and Rescue (SAR) officers in determining the probable search area of a missing aircraft using its Last Known Position. The system automatically calculates the possible aircraft position, search radius and area, desired area coverage, probability zones, the recommended search pattern and divided search sectors, and visualises everything on an interactive GIS map.

- **Possible aircraft position** from heading, speed, wind drift and elapsed time
- **Search radius / area** from aviation search formulas with weather expansion
- **Probability zones** (High / Medium / Low) rendered Red / Orange / Yellow
- **Recommended search pattern** via rule-based engine (5 ICAO patterns)
- **Search sectors** (A, B, C, D…) with colour coding and aircraft assignment
- **Professional PDF reports** with mission, map, weather and coverage data

2. Technology Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn-style UI, React Router, Framer Motion
Maps	Leaflet + React-Leaflet, OpenStreetMap (street / satellite / terrain / dark)
Charts	Chart.js + react-chartjs-2
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0, Pydantic v2, Uvicorn
Database	PostgreSQL (Railway) with optional PostGIS; SQLite for local dev
Authentication	JWT access tokens, bcrypt password hashing
PDF Reports	ReportLab + Matplotlib
Hosting	Vercel (frontend), Railway (API + database), GitHub (source)

3. System Modules

3.1 Authentication & Roles

- JWT login with bcrypt-hashed passwords and configurable token expiry
- Role-based access control: **admin** vs **sar_officer** on API and UI routes
- Admin credential rotation endpoint (**PATCH /users/{id}**) with uniqueness check

3.2 Dashboard

- Total / active / completed missions, aircraft missing today
- Live map preview of active missions, weather widget, recent-activity feed, notifications

3.3 Mission Management

- Create / update / pause / complete / delete with full validation (18 input fields: mission, aircraft, flight, registration, pilot, airports, LKP lat/lon, altitude, speed, heading, last-contact time, fuel, wind, weather,

visibility, remarks)

- Mission timeline, history, search + filter, activity timeline and logs
- CSV export, PDF export and email delivery of reports

3.4 Search-Area Calculation Engine

- Distance = Ground Speed × Elapsed Time
- Radius = distance + wind correction + navigation uncertainty (5% + 2 nm) + pilot deviation (10%)
- Severe weather expands uncertainty (×1.6); radius clamped to a 5 nm minimum
- Circular + polygon (GeoJSON) search regions with desired-coverage computation

3.5 Probability Engine

- Heading / wind / elapsed-time / speed / terrain weighted distribution around the predicted position
- High (red) / Medium (orange) / Low (yellow) zones rendered as map heat layers

3.6 Search Pattern Module

Condition	Recommended Pattern
High LKP accuracy	Expanding Square Search
Route known / tracked	Creeping Line Search
Very large search area	Parallel Track Search
Low confidence / uncertain	Sector Search
Well-defined flight path	Track Line Search

Patterns are drawn as waypoint polylines over the map.

3.7 Sector Division & GIS Map

- Automatic split into colour-coded Sectors A, B, C, D with per-sector aircraft assignment
- LKP marker + aircraft icon, radius circle, search polygon, probability heat zones
- Nearby-airport overlay, layer switcher, grid graticule, zoom controls, dark mode

3.8 Bonus Intelligence Features

- Crash-location prediction (heading + wind-drift modelling)
- Weather-impact prediction, mission risk analysis with rating
- Automatic search-priority ranking and future search-area simulation

4. Database Schema

Ten tables. Coordinates are stored as portable floats plus GeoJSON; PostGIS **geometry(Point/LineString/Polygon, 4326)** columns are provisioned automatically when available (optional).

Table	Purpose
users	Accounts, roles, bcrypt hashes
aircraft	Search-aircraft catalog (ALH Dhruv, Dornier 228, …)
missions	Mission inputs + computed radius, area, risk, predicted position

Table	Purpose
search_areas	Search polygon GeoJSON + coverage
search_patterns	5 candidate patterns with waypoints, recommended flag
probability_zones	High / Medium / Low bands with colours
sectors	Sector polygons, colours, assigned aircraft
reports	Generated PDF metadata
weather_snapshots	Stored Open-Meteo observations per mission
activity_logs	Audit trail of mission events

5. Key API Endpoints

Base URL: **/api/v1**. Interactive docs at **/docs**.

Method & Endpoint	Description
POST /auth/login	Login → JWT (admin / sar_officer)
POST /auth/register	Create user (admin only)
GET /dashboard/stats	Mission counters + missing-today
GET /dashboard/activities	Recent activity feed
GET+POST /missions	List / create mission
GET+PATCH+DELETE /missions/{id}	Mission CRUD (delete: admin)
GET /missions/{id}/search-area	Radius + polygon + coverage
GET /missions/{id}/probability	High/Medium/Low zones
GET /missions/{id}/pattern	5 patterns, recommended flagged
GET /missions/{id}/sectors	Sector polygons
PATCH /missions/sectors/{sid}	Assign search aircraft
GET /missions/{id}/analysis	Risk, crash prediction, simulation
GET /missions/{id}/simulation?hours=N	Area evolution forecast
POST /missions/{id}/reports	Generate PDF report
GET /missions/export/csv	CSV export
GET /weather/live?lat&lon;	Live Open-Meteo weather
PATCH /users/{id}	Profile + credential rotation (admin)

6. Security

- JWT access tokens with configurable expiry; bcrypt password hashing
- Role-based authorization on every sensitive route
- Pydantic validation on all inputs; SQLAlchemy parameterized queries (SQL-injection safe)
- CORS restricted to the deployed frontend origin
- Secrets via environment variables only — never committed (verified in git history)

7. Verification & Testing

- Backend: **46 pytest tests pass** (calculation, probability, pattern, sector, auth, API)
- Production end-to-end smoke test passed: login → create → radius/area/risk → zones → patterns → analysis → CSV → PDF → complete → delete
- Frontend: clean TypeScript build (**tsc + vite build**, 12.6 s) and live bundle verified
- Credential-rotation flow tested: weak rejected, old login dies, new login works, duplicates blocked

8. Running Locally

Docker (recommended): **cp .env.example .env && docker compose up --build -d** — frontend http://localhost:5173, API docs http://localhost:8000/docs. Manual: start PostGIS, **uvicorn app.main:app --reload** in server/, **npm install && npm run dev** in client/. Default bootstrap login is **admin / Admin@123** (change immediately; production credentials are rotated and held by the administrator).

8.1 Production Accounts

Production admin credentials were rotated from the defaults after deployment. Contact the system administrator for access.

— End of report — Made for safer skies. —